

E-Lab Notebook - Bleu Wilson

Tuesday, June 2, 2026

10:45 AM

Table of Contents

Team 1 - Quantum Background Literature

Week 1 - Getting Started

Background Literature (2026)

Zotero (Personal Research Assistant)

Research Topic: Quantum Neural Network's

Brain Chemistry Research

Week 2 - Defining a Topic

FFT's Topic Solidified

Week 3 -

Week 4 -

Week 5 -

Week 6 -

Week 7 -

Week 8 -

Team 1 - Quantum Materials

Tuesday, June 2, 2026 10:49 AM

Team 1 Focus: Computing, Computation, Quantum Materials

Expectations: This group will focus more specifically on:

- Computational Materials, E-chem and Materials Discovery
- HPC set-up, Quantinuum, Ion-Q, DWAVE Quantum and IBM partnerships
- Quantum Optics and Photonics
- Astrophysics and MSE curriculum development

Team Members: Bleu Wilson, Judi Bien-Amie, Julian Rus, Siobhan Rogers

Graduate Fellows/Mentors: Crossby Dessalines, Rick Mingione, Prof. Dikran Meliksetian, SCSU Faculty

Week 1 - Getting Started

Tuesday, June 2, 2026

12:10 PM

Day 1 (06/01) - Introductions

Day 2 (06/02) - Background Literature & NDA's

Helpful Materials:

- IBM Quantum Platform
- Notebook LM (Google)
- Colab (Google)

Day 3 (06/03) - Zotero Introduction

Literature Review/Research Question

Meet with Rick

View SEM

Day 4 (06/04) - Researching topics with Sadowski and broadbridge

Team 1 meetup & Creating research questions

Background information

Day 5 (06/05) - field trip to Wright Labs

Background Literature (2026)

Tuesday, June 2, 2026 10:49 AM

Computing, Computation, Quantum Materials

Mentor Suggested: IBM Quantum Platform - Foundations of Quantum Computing (Background)

Read up to Quantum Circuits

Read: Intro to Python (any platform)

Teams Chat:

Quantum

Materials Discovery

- [Lordi and Nichol - 2021 - Advances and Opportunities in Materials Science fo.pdf](#)
- [Paudel et al. - 2022 - Quantum Computing and Simulations for Energy Applications Review and Perspective.pdf](#)
- [Senjean et al. - 2023 - Toward density functional theory on quantum computers.pdf](#)

Review Articles

- [Krasnok - 2026 - What Quantum Computer to buy.pdf](#)
- [Ladd et al. - 2010 - Quantum computers.pdf](#)

Textbooks

- [Hughes - 2021 - Quantum Computing for the Quantum Curious.pdf](#)
- [Wong - 2022 - Introduction to Classical and Quantum Computing.pdf](#)

Review Articles

- [Chen et al. - 2024 - Accelerating Computational Materials Discovery with Machine Learning and Cloud High-Performance Comp.pdf](#)
- [Nwabara et al. - 2025 - High Throughput Computational and Experimental Methods for Accelerated Electrochemical Materials Dis.pdf](#)

Textbooks

- [Kaxiras and Joannopoulos - 2019 - Quantum Theory of Materials.pdf](#)
- [LeSar - 2013 - Introduction to Computational Materials Science Fundamentals to Applications.pdf](#)

Materials Characterization

Fuel Cells

Review Articles

- [Hussain and Yangping - 2020 - Review of solid oxide fuel cell materials cathode, anode, and electrolyte.pdf](#)
- [Ormerod - 2003 - Solid Oxide Fuel Cells.pdf](#)
- [Sharaf and Orhan - 2014 - An Overview of Fuel Cell Technology Fundamentals .pdf](#)

Textbooks

- [O'Hayre et al. - 2016 - Fuel Cell Fundamentals.pdf](#)

Microscopy

EBSD

Review Articles

- [Carneiro and Simões - 2020 - Recent Advances in EBSD Characterization of Metals.pdf](#)
- [Doddapaneni et al. - 2025 - Advancements in EBSD Techniques A Comprehensive Review on Characterization of Composites and Metals.pdf](#)
- [Stojakovic - 2012 - Electron Backscatter Diffraction in Materials Characterization.pdf](#)

FIB

Review Articles

- [Liu et al. - 2025 - FIB-SEM Emerging MultimodalMultiscale Characterization Techniques for Advanced](#)

[Battery Development.pdf](#)

- [Qiao and Zhang - 2026 - Recent Advances in FIB-SEM for Microstructural Characterization of Metallic Materials.pdf](#)

TEM

Textbooks

- [De Graef - 2003 - Introduction to Conventional Transmission Electron Microscopy.pdf](#)
- [Thomas and Gemming - 2014 - Analytical Transmission Electron Microscopy An Introduction for Operators.pdf](#)

Quantum Materials, EChem, Batteries

Electrochemistry

- [Elgrishi et al. - 2018 - A Practical Beginner's Guide to Cyclic Voltammetry.pdf](#)
- [Wang et al. - 2021 - Electrochemical Impedance Spectroscopy.pdf](#)

Textbooks:

- [Bard and Faulkner - 2001 - Electrochemical Methods Fundamentals and Applications.pdf](#)

Energy Storage

Batteries

Recycling

- [Baum et al. - 2022 - Lithium-Ion Battery Recycling—Overview of Techniques and Trends.pdf](#)

Review Articles

- [Go et al. - 2023 - Atomic Layer Deposition for Thin Film Solid-State Battery and Capacitor.pdf](#)
- [Sharma et al. - 2023 - Computational Materials Discovery and Development for Li and non-Li Advanced Battery Chemistries Re.pdf](#)

Textbooks

- [Kaxiras and Joannopoulos - 2019 - Quantum Theory of Materials.pdf](#)

IBM Quantum Platform (Review & Questions):

Overview: Basics of Quantum Information

Quantum Mechanics is the branch that allows us to describe things that happen at the atomic and subatomic level.

Mathematic description Quantum information for single and multiple systems, Quantum Circuits which describes quantum computations, phenomenon of quantum entanglement: Q teleportation, superdense coding, CHSH game/inequality.

CHSH inequality:

Single system quantum:

What the heck does quantum mean?

Quantum information is an extension of classical information.

- The **simple description** of quantum information states that it is simpler and typically learned 1st and quantum states represented by vectors; operations are represented by unitary matrices. Sufficient for understanding most quantum algorithms.
- The simplified description of quantum information can be viewed as a special case of the **general description** quantum information in the general sense is more broadly applicable quantum states represented by density

matrices; Allows for a more general class of measurements and operations. Quantum information in the general sense includes both the simplified description and the classical information including probabilistic states as special cases.

- Essentially, what this kind of looks like so far is classical information states regarding quantum is sort of representative of the visual state the information you're given I guess as opposed to the non visible state we can follow up on that later.

OK is it dictating what I'm saying right now? Nice, so basically this video is going over single system quantum understanding quantum information and computation and it is an hour long but I like that I can talk to my computer and it'll write stuff for me actually I should try this in like the Google Docs or word so instead of actually having to like write out every single word I want to do I can just sort of talk to my computer and say what I want to write and then it'll just write it that's kind of funny.

What is Quantum/What does Quantum mean?

Zotero (Personal Research Assistant)

Wednesday, June 3, 2026 9:33 AM

06/03

[Zotero.org](https://www.zotero.org)

Zotero Capabilities: Microsoft Teams without chat capabilities

Articles & Supplemental Information

Research Topic: QNN's

Wednesday, June 3, 2026 11:49 AM

Quantum Neural Networking (QNN)

What is a Quantum Neural Network?

Hybrid computational models that merge classical artificial neural networks with quantum mechanics.

Why do I want to focus on QNN's?

I would like to focus on QNN's because of an interest in Biomedical research, technology and engineering, specifically in regards to the neural networking fields and applications regarding future research.

Talking with Rick gave an insight in how to approach said field. I may not be able to go directly into QNN's themselves, especially with my own limited knowledge, along with the limited information in the field right now, but I can research on the development of QNN's and begin to look at potential future processes.

Starting Point: Exploring Hybrid quantum/classical computing in neuroscience

EEG's - Electroencephalogram

Brain Chemistry Research

Thursday, June 4, 2026 9:40 AM

The brain produces glucose/glutamate which during a brain scan the

Magnetoencephalography (MEG)

- **What it measures**
 -
- **Cost Accessibility**
 - Expensive
- **How this can relate to quantum?**
 - Quantum tech is foundation of both traditional and next gen MEG

Electroencephalography (EEG)

- **What it measures**
 - EEG's measure the electric/extracellular currents for radial and tangential brain activity.
- **Issues w/ Tech:** Signals often get distorted/*attenuated* during the process of passing through the scalp and skull.
 - Attenuated: reduction in force, effect, value
 - *My Theory: so essentially, signal becomes much weaker leading to less effective signal outreach and collectable data.*
- **Cost Accessibility**
 - Common: Found in neurology wards of hospitals/clinics
- **How this can relate to quantum?**

06/04

You're not curing the disease, you are trying to deal with the factors that may contribute to the disease.

Some questions worth asking

Can Quantum Computing be used in regards to Neuroscience?

How can Quantum be used to improve/expand on neurotechnology?

How does Quantum Computing Relate to Neuroscience?

06/05

Today's idea consist of modeling memory association using qubit which is supposed to be simple for people new to quantum or don't quite understand the concepts and the focus is on seeing if a computer can sort of let me look at it one second..

- **The Refined Question:** How can quantum entanglement be used to model basic memory association (e.g., how the smell of a flower triggers the memory of a garden) using a 3-qubit system?
- **Why it's friendly for beginners:** This replaces complex neurobiology with a simple logic puzzle. If Qubit A (Smell) is entangled with Qubit B (Sight), changing the state of Qubit A automatically changes Qubit B. The student is just learning the physics of entanglement and using "memory" as a fun, conceptual metaphor for it.

- **8-Week Deliverable:** A working Qiskit or PennyLane circuit showing that measuring one "sensory" qubit correctly outputs the state of the associated "memory" qubit.

Dictation: Essentially what we can look at is how memory association is modeled as of right now and according to Google yeah a lot of memory association and there is an intersection of neuroscience and artificial intelligence which relies heavily on like mathematical models to show how memory is directly associated to maybe smells or visuals using vector embedded techniques Hopfield networks and predictive coding I think that predictive coding might be closest to what we're looking with with the three bit qubit system as a logic puzzle umm so a biological and predictive coding model simulate specific brain regions to explain how humans contextualize memories using predictive coding these networks generate expectations based on prior experience and memory recall occurs when sensory data updates or corrects internal predictions. So essentially this model would explain how some sort of cue like a smell or maybe a visual can you're a complex multi sensory recall. Another thing to look at would be transformer architectures since those are based heavily off of AI and right now in it it sort of looks like that possibly people are grouping and AI together to use one in order to assist the other so because we're not really clear on what quantum does yet or what it's capable of doing people want to use AI to sort of either assisted or just because it's very hot right now use it in association too the AI models large language models umm.

Thought process 1: if we're looking at the eight week deliverable working with qiskit showing that measuring A sensory qubit correctly outputs the state of an associated memory qubit, then we can figure out the different states of associated memories. Are they as simple as on and off (like if you smell something in it relates to a memory you had does that mean that the associated memory is on refuse smell something and it doesn't relate to that memory it's off)? like what state is the memory itself in and how can it detect that that's actually a memory? are we looking at umm brain activity like are there certain parts of the brain connected to memory that will fire off electric signals or even maybe magnetic signals of some sort? for looking at quantum entanglement to show that we don't know the exact memory it relates to but it does relate to something in the memory. but then that brings up a new question: is it a memory we know or is the brain itself just firing off from that section? like can you visualize the memory or when that trigger happens, is that part of the brain just active either way whether it's actually a memory or not?

06/08

AI is representative of semantic memory

Episodic memory is sensory memory that delivers you back to a certain place. Episodic memory is likely what we will focus on in regard to basic memory modeling. Episodic memories are diverse based entirely on individual experience.

The hippocampus is in charge of memory-based brain functions

Video: [Adaptive Memory: How the Brain Uses Prediction to Shape How We Remember](#)

How is FFT used to map brain scan and applied to Quantum

Week 2 - Defining a Topic

Tuesday, June 2, 2026

12:11 PM

Day 6 (06/08) - FFT's Solidified Topic (Quantum Fast Fourier Transforms)

Day 7 (06/09)- Project Proposal

Presentations

Day 8 (06/10)- Project Proposal

Project Framework w/ Rick

Coding

Day 9 (06/11) - Project Proposal

qBraid Coding Practice w/ Sadowski

Day 10 (06/12)- YQI Labs

FFT's Topic Solidified

Monday, June 8, 2026 11:45 AM

Dictation: *OK, so today I met with doctor Broadbridge and professor Sadowski and we talked about my final topic (or sort of more solid topic) that I will present tomorrow on Fast Fourier Transforms and how they relate to neuroscience and neurotechnology using quantum based coding and quantum technologies. so now I just have to look into that a little bit more and we should be able to find something more clear now that I understand what direction I'm actually moving with this which is a lot simpler I think to do.*

Fast Fourier Transforms:

What is an FFT?

How Do FFT's Work?

What platforms will we use?

How to write this in a way for future readers to understand?

Question - What are the most important parameters for optimization of Quantum FFT code for successful implementation on quantum computers in neuroscience? (Credits: Sadowski, T.)

An FFT is

Breakdown:

Fast: moving or capable of moving at high speed.

Fourier: the analysis of a periodic function into its simple sinusoidal or harmonic components, whose sum forms a Fourier series.

Fourier Series: a mathematical method that breaks down any complex, repeating (periodic) waveform into an infinite sum of simple, smooth sine and cosine waves

Transform: the product of a transformation.

Discrete: individually separate and distinct.

Basic Starting Definition: A fast moving analysis of the transformation of complex periodic waveforms into a product of more basic finite sine/cosine waves.

AI Definition: FFT: *A highly optimized computer algorithm that rapidly transforms a finite chunk of a complex, sampled waveform into its component frequencies, revealing the exact recipe of specific sine and cosine waves that create it.*

QFFT's: QFFT's are specific quantum circuits designed for the purpose of executing classical FFT's

Platforms: I cant remember what they said today

- FFT's, QFT's, QFFT's
- **Mechanism:** QFT operates on the basis states of qubits via superposition and quantum interference. QFFT seeks to optimize and map the classical Fast Fourier Transform (FFT) logic onto quantum hardware to process superpositioned data streams simultaneously. [[1](#), [2](#)] (Credit: Google AI).
- **Use Cases:** QFT is the foundational subroutine for core quantum algorithms like Shor's (for factoring and cryptography) and Quantum Phase Estimation (QPE). QFFT is being researched to enhance quantum machine learning, high-dimensional data compression, and image processing. [[1](#), [2](#), [3](#), [4](#)] (Credit: Google AI).

- **Complexity:** Exact QFTs require very deep circuits and precise, increasingly small phase rotations that are prone to decoherence on noisy hardware. QFTs are often explored as algorithmic approximations to make these circuits more scalable. [\[1\]](#) (*Credit: Google AI*).

To be specific, we are looking at QFFT's (aka QDFT's or Quantum Discrete Fourier Transforms).

Presentation (Initial)

Monday, June 15, 2026 11:08 AM

https://owlssouthernct.sharepoint.com/:p:/r/sites/TMSWerthIAFParticipants/_layouts/15/Doc.aspx?action=edit&sourcedoc=%7Ba359a6bf-b35e-43c2-9f60-c509b0231536%7D&wdExp=TEAMS-TREATMENT&web=1&TeamsCID=9b0a592f-89e1-4d57-9a16-420accf34b78



Project Proposal (Initial)

Tuesday, June 9, 2026 1:50 PM

Quantum in itself is an inherently broad topic. It could mean and refer to almost anything, as everything in the universe is connected to quantum to a degree. To simplify this topic a bit more, first a framework needs to be created to achieve specific proposed ideas which utilize quantum. In the case of medical field related quantum, it must be narrowed down even further, seeing as medical applications span a wide gap. Regarding the relationship between neuroscience and quantum computing, first we must identify key parameters in order to optimize Quantum Fast Fourier Transform code to then successfully implement onto quantum computers.

Location: Microsoft Word (Wilson - Project Proposal)/Teams - General - 2026 - Deliverables - Project Proposal - Papers - Team 1

Optimizing Fast Fourier Transforms for Quantum Neuroscience

Bleu Wilson¹, Thomas Sadowski¹, Rich Pellegrino¹, Christine C. Broadbridge¹

¹Werth Industry Academic Fellowship, Southern Connecticut State University, New Haven, CT 06515, USA

Abstract (06/09)

In regard to neuroscience, Fast Fourier Transforms (FFT) are a mathematical tool that in a way untangle messy, overlapping brain signals into distinct components like trigonometric wave functions. By applying this concept to Quantum Fast Fourier Transforms (QFFT) (separate from QFT), basis encoding can be used to translate precise brain measurements into a quantum format that is significantly more powerful than the use of classical methods. This approach is potentially more effective because it uses quantum parallelism to process multiple electroencephalograph (EEG) channels or larger datasets simultaneously in a state of superposition, rather than analyzing them one by one as classical computers would do. Essentially, quantum FFT's could not only fast track this process, but are capable of running data that classical computers have not been able to solve clearly yet.

Introduction (Motivation and Background) (06/09)

Quantum is a delicate computer facing phenomenon which is heavily impacted by the physical world around it. In itself, Quantum is such a broad word. When describing quantum phenomenon, it is best to start with the particular subject matters pertaining to quantum phenomenon. Quantum entanglement, superposition, quantum measurement, quantum interference; these are all subtopics of quantum, used to describe and explain other subtopics—it's very confusing. Because one phenomenon cannot simply be explained without using another, the definitions of different quantum subtopics seem to be entangled within themselves. The textbook definition which describes superposition, met with entanglement, is definitively relative; or in laymen's terms, it is conditional.

Quantum in itself is an inherently broad topic. It could mean and refer to almost anything, as everything in the universe has some degree of quantum connection. To simplify this topic a bit more, first a framework needs to be created to achieve proposed ideas which specifically utilize quantum computing. In the case of medical field related quantum, it must be narrowed down even further, seeing as medical applications span a wide

gap. Regarding the relationship between neuroscience and quantum computing, first we must identify key parameters in order to optimize Quantum Fast Fourier Transform code to then successfully implement onto quantum computers.

To be clear, this is a process that is not completely straightforward. Summarizing quantum computing briefly: Quantum Computing is a specialized field which combines both physics and computer science to utilize the principles of quantum mechanics specifically regarding superposition, entanglement, and interference with perform computations that can overcome the limitations of classical computing [1][6]. A good understanding of this will leave a steady base for understanding the core components of optimizing any quantum system; input, constraints, and outcome or results.

In terms of motivation behind this specific project, there is a clear focus on FFT's and improving the medical field, particularly for the brain. The basic idea is that by utilizing specialized quantum memory systems, the time it takes to encode brain data can be reduced significantly, which happens to be the motive behind this project. This could possibly open the door for quantum computers to identify complex neural patterns at speeds that are impossible for even the fastest of classical computers.

Proposed Methods (06/10)

This proposal is framed around a comparative, parameter driven literature analysis. To keep things simple, a possible method going forward is to use a systematic parameter guided theoretical evaluation of quantum fast Fourier transforms, comparing how classical EEG datasets are handled across two primary quantum computing models. To run successfully on quantum hardware, the code is built into simple arithmetic steps, such as addition and subtraction, and carefully designed to be reversible and free of bits that would otherwise cause errors. For this research topic, there will be a specific focus on neurology. Using something called Fast Fourier Transforms (aka: Discrete Fourier Transforms); which are a form of classical systems already well used in the medical field to process electrical signal data from the brain, these can be converted to Quantum Fast Fourier Transforms to expedite the process. First, it will be important to quantify algorithmic resource parameters.

In the given time, it will be important to deduce, contrast, and map the key architectural constraints of the two main quantum methods found within the literature. The first is an amplitude encoded QFT [5][8]; which will map data onto the amplitudes (aka the probability states) of entangled qubits. The measured parameters will calculate the required qubit count ($n = \log_2(N)$) and gate depth for standard EEG windows (such as 256 samples require 8 qubits).

The second main quantum method is the register-based classical FFT circuit [4]. This particular method maps data onto a tensor product of quantum states, implementing quantum arithmetic gates (adders, subtractors, and bit-shifts) to mirror the classical FFT workflow without generating extra noise. The measured parameters should evaluate data storage versatility and the scaling of arithmetic gates.

The second piece regards parameter tradeoff mapping for neuroscience data. This will analyze how changing the input parameters of raw brainwave data; specifically, the 1-second/256 Hz baseline [2], impact the physics of a quantum computer. This will mathematically evaluate the parameter thresholds for gate truncation & coherence Boundaries: Comparing calculated circuit runtimes against basic dephasing limits (T_{times}) [6] to see if the calculation finishes before the qubits drop from their quantum state.

Expected Results and Discussion (06/11)

By the end of this, the hope is to see a link between quantum computing to neuroscience, specifically seeing how quantum code utilizing Quantum Fourier Transforms can potentially optimize signal processing. Within this, there are a few key results to be viewed. The most limiting parameter in regard to the hardware is depth of the circuit relative to coherence time. A 1-second window (256 samples) is optimal for real-time brainwave tracking [2]. Environmental noise causes qubits to decay rapidly (decoherence) [6]. The analysis is expected to show that while an 8-qubit Standard QFT [5] handles a 1-second EEG window easily, the required sequence of Hadamard and controlled-phase gates will exceed the coherence limits of noisy intermediate-scale quantum devices unless approximate, truncated gate limits are introduced.

The second potential result is finding the ultimate barrier to optimization is the State Preparation Overhead Parameter. The mathematical speedup of the QFT is exponential ($O(\log N)$) because it acts on cyclic groups natively [7]. However, a particular issue of transforming classical EEG voltage numbers into quantum amplitudes requires a long, complex data loading step. The code must be optimized to only yield true if the brainwave data remains in a quantum state after the transform to be processed by an on-chip Quantum Neural Network.

After finding the correct code to use, it's important to consider the sheer amount of resources that will be used up. While a register-based FFT circuit keeps data perfectly preserved and structured like classical EEG files, it requires a massive amount of physical qubits compared to the amplitude encoded QFT [4].

Proposed Timeline (06/11)

As a complete beginner in this research topic, it is hard to estimate an exact timeline for everything to happen. Given the rough six weeks left to work on this project, the hope is to at least get some insight into what parameters fit for this. Personal challenges and limitations may include re-learning how to code, learning new software such as IonQ or Qiskit, and partial limitations regarding access (from the developer's part).

After consulting a mentor and a large language model* for help structuring a reasonable timeline, an estimated 1-2 week projection was placed. The first few days of week one should consist of consulting the literature to find specific formulas and data encoding frameworks to map how classic EEG signals fit into the equations. In the second half of week one, the process will consist of figuring out the mathematical steps used to convert FFT's into quantum circuits, along with identifying where gates could be removed or compressed. In week two, the limitations and bottlenecks of the overall process can be mapped out. The project estimation is only 1-2 weeks, so time may be allocated in case of errors or extended time is needed during pieces of the process. This is to ensure that this project can be completed within the 6-week limit, and there will be some amount of results to show by the ending date.

Acknowledgments

The author acknowledges primary support from the Werth Endowed Nanotechnology Industry Academic Fellowship (IAF) Program with additional support from the CT State College and Universities Center for Nanotechnology and SCSU Innovation HUB. The Author would like to specifically thank Dr. Christine Broadbridge, Professor Thomas Sadowski, Rich Pellegrino, and Rick Mingione for providing this

opportunity along with their constant support and understanding regarding the provided research topic.

References

- [1] Oxford Languages.
- [2] Keller. AJ, "The Fast Fourier Transform for EEG." *Neurocity*, Jan 2026, <https://neurocity.co/guides/fast-fourier-transform-fft-eeg-analysis-primer>.
- [3] Marquezino. F.L, et al. "Obtaining the Quantum Fourier Transform from the classical FFT with QR decomposition." *Sciencedirect*, 235, Nov 2010, <https://www.sciencedirect.com/science/article/pii/S0377042710002888>
- [4] Asaka. R, et al. "Quantum Circuit for the Fast Fourier Transform." *Springer Nature Link*, 19, 277, Aug 2020, <https://link.springer.com/article/10.1007/s11128-020-02776-5>.
- [5] Zilliz, "How does a quantum computer perform a Fourier transform?" *Milvus AI Reference*, 2026. <https://milvus.io/ai-quick-reference/how-does-a-quantum-computer-perform-a-fourier-transform>.
- [6] C. Hughes, J. Isaacson, A. Perry, R.F. Sun, and J. Turner, "Quantum Computing for the Quantum Curious" *Springer Nature*, 2021.
- [7] M. Schuld, "It's all about groups: From Fast Fourier Transforms to QFTs," *PennyLane Demos*, April 2026. https://pennylane.ai/demos/tutorial_qft_and_groups.
- [8] T.G. Wong, "Introduction to Classical and Quantum Computing." 2022.

**Google Gemini and NotebookLM were used to help format this proposal and help focus a topic on a cohesive point. All words and ideas in writing are my own. Any ideas or proposals are subject to change over the duration of this project.*

Week 3

Tuesday, June 2, 2026

12:11 PM

Day 11 (06/15) - Prompts for FFT Project

7-Day Action Agenda

Day 12 (06/16) - Project Code

Day 13 (06/17) -

Day 14 (06/18) -

Day 15 (06/19) -

Prompts for FFT Project

Monday, June 15, 2026 9:42 AM

Location: Gemini - Quantum & the Brain

Are there any cross-sections between quantum Fourier transforms and [your topic here]?

- Based on the response you might need to reframe to come to some topic that seems feasible.

Is there a simple research question that I could adopt and build an experiment to test within a week?

Please sort your response from easiest to hardest for coding difficulty.

- This prompt gave me a number of options to choose from.

Let's focus on Experiment 1. Provide me an agenda on how to complete this for experimentation within 1 week time. Please provide daily actions and layout the full experiment step by step.

- This gave me an itinerary I could build off of. I understood the basic building blocks of what I needed to make after this response.

If I want to build this project what kind of plots, graphs, or other metrics can I output that would support a research presentation.

- Let it give you a list and then implement which seem appropriate

Build the code base for this project. Include the file structure and then break the code into components. Use python as the coding language.

- This should output you the code and you want to copy and paste into the correct file types. I suggest downloading vs code so use for looking at your code and running the scripts.

Write me a README.md with instructions on how to run this experiments and then commands I need to perform in which order.

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Here is a simple, lightweight template using Python and Qiskit to simulate the standard Quantum Fourier Transform (QFT). This serves as the basic code blueprint for your 1-to-2 week proposal, demonstrating how classical data lengths translate into quantum qubits and gates

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

```
def apply_qft(circuit, n):
    """Applies a standard Amplitude-Encoded QFT to a circuit loop."""
    if n == 0:
        return circuit
    n -= 1
    circuit.h(n) # Apply Hadamard Gate
    for qubit in range(n):
        # Apply Controlled-Phase Rotations
        circuit.cp(np.pi / 2**(n - qubit), qubit, n)
    # Recursively build the rest of the circuit
    apply_qft(circuit, n)

# --- 1. DEFINE BIOLOGICAL PARAMETERS ---
# Neurosity Crown baseline: 1-second window at 256 Hz = 256 classical samples
samples = 256
num_qubits = int(np.log2(samples)) # Requires exactly 8 qubits

# --- 2. INITIALIZE QUANTUM CIRCUIT ---
qft_circuit = QuantumCircuit(num_qubits)

# --- 3. RUN THE QFT ALGORITHM ---
apply_qft(qft_circuit, num_qubits)

# --- 4. SIMULATE IN A NOISELESS ENVIRONMENT ---
simulator = AerSimulator()
compiled_circuit = transpile(qft_circuit, simulator)

print(f"Success: Generated QFT circuit for {samples} EEG samples using {num_qubits} qubits.")
print(f"Total Circuit Depth (Gate Layers): {qft_circuit.depth()}")
```

How to use this in your proposal draft:

- For the Methods Section:** You can reference this exact code structure to show that you are testing an 8-qubit system to transform 256 brainwave data points.
- For the Limitations Section:** Point out that AerSimulator() runs in a perfect, noiseless environment. Your proposal's core analysis focuses on what happens when you introduce realistic hardware noise (decoherence) to those gate layers.

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Prompt: "Okay, so using the research question I provided earlier ("What are the most important parameters for optimization of Quantum FFT code for successful implementation on quantum computers in neuroscience?"), can I build an experiment to test within 1-2 weeks? Please sort your response from easiest to hardest for coding difficulty."

Yes, you can absolutely build a functional experiment within 1–2 weeks! Since you are working in a simulator (like AerSimulator on qBraid), you don't have to worry about physical hardware wait times or expensive cloud costs. You can simulate the entire pipeline from raw brain data to quantum output.

To achieve this in your timeframe, the strategy is to use **synthetic or small-scale open-source EEG data**, truncate the signal lengths to keep the required qubit count low, and benchmark how changing specific quantum circuit parameters affects the accuracy and performance of the Fourier transform.

Here is a roadmap of experimental designs you can build, ordered from **easiest to hardest in terms of coding difficulty**.

Experiment 1: The "Small Window" Gate-Truncation Experiment

- Core Concept:** Standard QFT requires $O(n^2)$ gates because it applies controlled-rotation (R_k) gates between qubits. When the distance between qubits is large, the rotation angle becomes so microscopic that it contributes almost nothing to the final result but adds massive noise and circuit depth. This experiment tests the impact of cutting off (truncating) small rotations.
- Neurology Context:** Real-time Brain-Computer Interfaces (BCIs). BCIs require incredibly fast processing. Can we strip away minor quantum gates to speed up execution without losing the ability to detect alpha or beta brainwaves?
- How to Code It:**
 - Generate a synthetic EEG signal (e.g., a combination of a 10 Hz alpha wave and a 20 Hz beta wave).
 - Encode a small window of this data (e.g., 8 or 16 data points) into a 3-qubit or 4-qubit quantum state using **Amplitude Encoding**.
 - Write a custom QFT function where you include an adjustable parameter `bandwidth_cutoff`. If an R_k gate's rotation angle is smaller than this cutoff, the code skips it.
 - Compare the resulting quantum frequency spectrum against a classical FFT.
- What you measure (Optimization Parameters):** Circuit Depth vs. Frequency Reconstruction Accuracy.
- Why it's Easiest:** Qiskit has built-in QFT modules, but writing a modified loop for a 3-to-4 qubit circuit is only a few lines of basic Python.

Experiment 2: State Preparation & Data Encoding Bottleneck Test

- Core Concept:** One of the biggest traps in Quantum Information Processing is the "state preparation problem." QFT is exponentially fast, but loading classical data (like a time-series voltage log from an EEG) into a quantum state can take exponential time ($O(2^n)$ gates), completely destroying your quantum speedup.
- Neurology Context:** High-Density EEG or MEG signal streaming. If a sensor array is streaming brain data at 500 Hz, the quantum computer must ingest it faster than it arrives.
- How to Code It:**
 - Take a fixed-length segment of EEG data.
 - Code three different ways to load this data into Qiskit:
 - Basis Encoding:** (Converts values to binary strings; requires many qubits).
 - Angle Encoding:** (Encodes data points

Prompt: "Because I'm fairly unfamiliar with quantum coding, let's focus on Experiment 1. Provide me an agenda on how to complete this for experimentation within 1 week time. Please provide daily actions and layout the full experiment step by step. (Please keep experiment 3 in mind for a future project)."

To complete **Experiment 1 (The Gate-Truncation Experiment)** in one week on qBraid, we will break the process down into highly manageable, daily steps. The goal of this experiment is to see if we can optimize a Quantum Fourier Transform circuit for real-time neurology applications (like Brain-Computer Interfaces) by stripping away microscopic, noisy quantum gates without losing our ability to detect specific brainwave frequencies.

Here is your step-by-step experiment layout and your daily action plan.

The Experiment Layout (Step-by-Step)

1. **The Biological Signal:** Generate a synthetic EEG data stream containing two distinct brainwave frequencies (e.g., a **7 Hz Theta wave** representing a relaxed/meditative state and a **15 Hz Beta wave** representing active thinking).
2. **Data Pruning:** Take a tiny slice of this continuous signal (8 data points) so it perfectly fits onto a **3-qubit** circuit ($2^3 = 8$).
3. **Quantum Ingestion:** Load these 8 classical data points into the amplitudes of our 3 qubits using Amplitude Encoding.
4. **The Variable (Truncation):** Run two versions of the QFT circuit:
 - **Control Group:** A full, standard QFT circuit that applies every single rotation gate.
 - **Experimental Group (Truncated QFT):** A modified QFT where we delete the smallest controlled-phase rotation gate (which adds circuit depth but contributes very little mathematical value).
5. **The Analysis:** Measure the qubits, get the frequency outcomes, and evaluate if the Truncated QFT can still successfully tell the difference between the 7 Hz and 15 Hz brainwaves.

Your 7-Day Action Agenda

Day 1: Set Up your Environment & Generate Brainwaves

- **Goal:** Set up your notebook on qBraid and create your fake EEG data.
- **Actions:**
 1. Open a new Jupyter Notebook on qBraid using a Python environment that has qiskit and qiskit-aer installed.
 2. Use standard Python libraries (numpy and matplotlib) to generate a synthetic EEG time-series wave. Blend a 7 Hz wave and a 15 Hz wave together.
 3. Plot this raw wave to make sure it looks like a messy, organic brainwave trace.

Day 2: Prepare Data for the Qubits (State Preparation)

- **Goal:** Truncate your continuous data slice so the quantum circuit can read it.
- **Actions:**
 1. Sample exactly 8 points from your wave.
 2. Normalize the vector (divide the values so the sum of their squares equals 1). This is a mathematical requirement because quantum probabilities must always add up to 100%.
 3. Look up Qiskit's StatePreparation or qc.initialize() function. Practice writing a script that takes your 8 numbers and loads them into a blank 3-qubit circuit.

Day 3: Code the Standard QFT (The Control Group)

- **Goal:** Build the complete, un-optimized QFT circuit.
- **Actions:**
 1. Instead of using Qiskit's prebuilt QFT feature, manually code a 3-qubit QFT using loops or individual gates. This is crucial because you need to control the individual gates.
 2. A 3-qubit QFT consists of Hadamard (H) gates and Controlled-Phase (\$CP\$) gates. For 3 qubits, your rotation angles will be $\pi/2$ and $\pi/4$.
 3. Print the circuit diagram using qc.draw('text') to visually verify the structure.

Day 4: Code the Truncated QFT (The Experimental Group)

- **Goal:** Create your optimized, shallow-circuit variation.
- **Actions:**
 1. Duplicate your QFT code from yesterday.
 2. Locate the smallest rotation gate in your 3-qubit design—this will be the $CP(\pi/4)$ gate acting between qubit 0 and qubit 2.
 3. Comment out or delete that specific gate. By removing it, you have drastically reduced the "depth" of your circuit (making it faster and less susceptible to the hardware noise you'll test in Experiment 3 later).

Day 5: Run the Simulations

- **Goal:** Execute both circuits and collect your frequency data.
- **Actions:**
 1. Attach a measurement gate to the end of both your Control circuit and your Truncated circuit.
 2. Initialize the AerSimulator().
 3. Run both circuits for 1024 shots. Save the results into two separate dictionaries: control_counts and truncated_counts.

directly into rotation angles θ of R_X gates; requires 1 qubit per data point, execution is $O(1)$.

- **Amplitude Encoding:** (Encodes the entire wave shape into the state vector amplitudes; highly qubit-efficient but structurally complex).
- 3. Run Qiskit's transpiler (transpile(circuit, backend)) to see how many raw physical gates each encoding method generates.
- **What you measure (Optimization Parameters):** State Preparation Gate Count vs. Qubit Economy.
- **Why it's Moderate:** You don't have to code complex math; Qiskit has native features like Initialize() or StatePreparation classes, but you will need to learn how to manipulate data structures and parse circuit statistics (using circuit.count_ops()).

Experiment 3: Mitigation of Hardware Noise on Biological Frequency Bands

- **Core Concept:** Real quantum computers are noisy. This experiment introduces simulated hardware noise (thermal relaxation and gate errors) to see at what point a quantum computer can no longer reliably distinguish between different mental states (e.g., sleep vs. highly focused).
- **Neurology Context:** Clinical Diagnostics (detecting epileptic spikes or sleep stages). If quantum hardware noise mimics biological artifacts, a doctor could get a false reading.
- **How to Code It:**
 1. Build a standard QFT circuit in Qiskit.
 2. Use qiskit_aer.noise to construct a simulated noise model based on a real IBM Quantum backend (like ibm_brisbane).
 3. Feed two distinct synthetic signals into your QFT: one representing a calm state (dominated by 10 Hz Alpha waves) and one representing a deep sleep state (dominated by 2 Hz Delta waves).
 4. Run the simulation across varying levels of noise and count how many "shots" (repeats) are necessary to clearly separate the peak of the Alpha wave from the Delta wave.
- **What you measure (Optimization Parameters):** Required Shot Count vs. Backend Error Rate (Fidelity limits).
- **Why it's Harder:** It requires importing and configuring Qiskit's Aer noise modules, which can have a steeper learning curve for beginners, and requires an understanding of how quantum probability distributions degrade under decoherence.

Experiment 4: The Tensor-Product FFT Circuit (The "No-Garbage-Bits" Approach)

- **Core Concept:** In a standard QFT, data is mapped directly to the amplitudes of a superposition. However, a major strain of academic research explores building a *literal* Fast Fourier Transform circuit using quantum arithmetic (quantum adders, subtractors, and multipliers) acting on a tensor product of register states rather than pure amplitudes.
- **Neurology Context:** Connectomics and Network Phase-Synchrony. When analyzing network graphs of billions of firing synapses, you need to preserve data paths exactly without destroying them via unmitigated measurement.
- **How to Code It:**
 1. Instead of using Qiskit's prebuilt abstract QFT gate, you must manually construct a quantum circuit that replicates the Cooley-Tukey butterfly algorithm matrix using basic arithmetic blocks.
 2. You will have to design or import a Quantum Adder (like a Draper Adder) to compute the combinations of your neural time-series inputs.
- **What you measure (Optimization Parameters):** Qubit Overhead (ancilla bits) vs. Algorithmic Versatility.
- **Why it's Hardest:** This requires advanced quantum circuit design. You cannot rely on high-level abstract functions. You are essentially building a microscopic calculator out of quantum logic gates from scratch. *Given your 1–2 week timeline, it is recommended to skip this one or leave it for future graduate-level work.*

Recommended Choice for your Project: Experiment 1 or 2

your Truncated circuit.

2. Initialize the AerSimulator().
3. Run both circuits for 1024 shots. Save the results into two separate dictionaries: control_counts and truncated_counts.

Day 6: Analyze & Plot the Results

- **Goal:** Determine if the optimization ruined the neuroscience data or saved it.
- **Actions:**
 1. Use Qiskit's plot_histogram to plot both outputs side-by-side.
 2. Look at the peaks. In a successful QFT, the states with the highest probabilities correspond directly to the dominant frequencies (your 7 Hz and 15 Hz waves).
 3. Check if your Truncated circuit from Day 4 still shows the peaks clearly, or if deleting that gate blurred the results into random noise.

Day 7: Document and Reflect (Preparing for Experiment 3)

- **Goal:** Tie the experiment back to your overarching research prompt.
- **Actions:**
 1. Write down your conclusion: *Did gate truncation preserve the brainwave features while lowering circuit depth?* (This directly answers what parameters are important for QFT optimization in neurology).
 2. Note how many gates you saved.
 3. Prepare notes for your future project: Because you now know how deleting gates changes the circuit's ideal performance, you are perfectly positioned to introduce a simulated IBM hardware noise model next time to see if this truncation trick actually protects the brainwave data from decoherence!

your 1–2 week timeline, it is recommended to skip this one or leave it for future graduate-level work.

Recommended Choice for your Project: Experiment 1 or 2

If you want a highly successful project completed smoothly in 2 weeks, **Experiment 1 (Gate-Truncation)** is the sweet spot. It directly answers the prompt: it optimizes a critical QFT parameter (rotation bandwidth/circuit depth), shows how that parameter affects a biological requirement (resolving specific brainwave frequencies), and can be written in less than 60 lines of cleanly commented Qiskit code on qBraid.

7-Day Action Agenda

Monday, June 15, 2026 10:56 AM

Location: Gemini - WerthIAF 2026 (Quantum Coding)

This Agenda will model Experiment 1 that Google Gemini Helped me create:

Prompt: "Because I'm fairly unfamiliar with quantum coding, let's focus on Experiment 1. Provide me an agenda on how to complete this for experimentation within 1 week time. Please provide daily actions and layout the full experiment step by step. (Please keep experiment 3 in mind for a future project)."

To complete **Experiment 1 (The Gate-Truncation Experiment)** in one week on qBraid, we will break the process down into highly manageable, daily steps.

The goal of this experiment is to see if we can **optimize a Quantum Fourier Transform circuit for real-time neurology applications (like Brain-Computer Interfaces) by stripping away microscopic, noisy quantum gates without losing our ability to detect specific brainwave frequencies.**

Here is your step-by-step experiment layout and your daily action plan.

The Experiment Layout (Step-by-Step)

- The Biological Signal:** Generate a synthetic EEG data stream containing two distinct brainwave frequencies (e.g., a **7 Hz Theta wave** representing a relaxed/meditative state and a **15 Hz Beta wave** representing active thinking).
- Data Pruning:** Take a tiny slice of this continuous signal (8 data points) so it perfectly fits onto a **3-qubit circuit** ($2^3 = 8$).
- Quantum Ingestion:** Load these 8 classical data points into the amplitudes of our 3 qubits using Amplitude Encoding.
- The Variable (Truncation):** Run two versions of the QFT circuit:
 - Control Group:** A full, standard QFT circuit that applies every single rotation gate.
 - Experimental Group (Truncated QFT):** A modified QFT where we delete the smallest controlled-phase rotation gate (which adds circuit depth but contributes very little mathematical value).
- The Analysis:** Measure the qubits, get the frequency outcomes, and evaluate if the Truncated QFT can still successfully tell the difference between the 7 Hz and 15 Hz brainwaves.

Your 7-Day Action Agenda

Day 1: Set Up your Environment & Generate Brainwaves

- Goal:** Set up your notebook on qBraid and create your fake EEG data.
- Actions:**
 - Open a new Jupyter Notebook on qBraid using a Python environment that has qiskit and qiskit-aer installed.
 - Use standard Python libraries (numpy and matplotlib) to generate a synthetic EEG time-series wave. Blend a 7 Hz wave and a 15 Hz wave together.
 - Plot this raw wave to make sure it looks like a messy, organic brainwave trace.

Day 2: Prepare Data for the Qubits (State Preparation)

- Goal:** Truncate your continuous data slice so the quantum circuit can read it.
- Actions:**
 - Sample exactly 8 points from your wave.
 - Normalize the vector (divide the values so the sum of their squares equals 1). This is a mathematical requirement because quantum probabilities must always add up to 100%.
 - Look up Qiskit's StatePreparation or qc.initialize() function. Practice writing a script that takes your 8 numbers and loads them into a blank 3-qubit circuit.

Day 3: Code the Standard QFT (The Control Group)

- Goal:** Build the complete, un-optimized QFT circuit.
- Actions:**
 - Instead of using Qiskit's prebuilt QFT feature, manually code a 3-qubit QFT using loops or individual gates. This is crucial because you need to control the individual gates.
 - A 3-qubit QFT consists of Hadamard (H) gates and Controlled-Phase (SCP) gates. For 3 qubits, your rotation angles will be $\pi/2$ and $\pi/4$.
 - Print the circuit diagram using qc.draw('text') to visually verify the structure.

Day 4: Code the Truncated QFT (The Experimental Group)

- Goal:** Create your optimized, shallow-circuit variation.
- Actions:**
 - Duplicate your QFT code from yesterday.
 - Locate the smallest rotation gate in your 3-qubit design—this will be the $\text{SCP}(\pi/4)$ gate acting between qubit 0 and qubit 2.
 - Comment out or delete that specific gate. By removing it, you have drastically reduced the "depth" of your circuit (making it faster and less susceptible to the hardware noise you'll test in Experiment 3 later).

Day 5: Run the Simulations

- Goal:** Execute both circuits and collect your frequency data.
- Actions:**
 - Attach a measurement gate to the end of both your Control circuit and your Truncated circuit.
 - Initialize the AerSimulator().
 - Run both circuits for 1024 shots. Save the results into two separate dictionaries: control_counts and truncated_counts.

Day 6: Analyze & Plot the Results

- Goal:** Determine if the optimization ruined the neuroscience data or saved it.
- Actions:**
 - Use Qiskit's plot_histogram to plot both outputs side-by-side.
 - Look at the peaks. In a successful QFT, the states with the highest probabilities correspond directly to the dominant frequencies (your 7 Hz and 15 Hz waves).
 - Check if your Truncated circuit from Day 4 still shows the peaks clearly, or if deleting that gate blurred the results into random noise.

Day 7: Document and Reflect (Preparing for Experiment 3)

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Experiment 1: The "Small Window" Gate-Truncation Experiment

- Core Concept:** Standard QFT requires (n^2) gates because it applies controlled-rotation (R_k) gates between qubits. When the distance between qubits is large, the rotation angle becomes so microscopic that it contributes almost nothing to the final result but adds massive noise and circuit depth. This experiment tests the impact of cutting off (truncating) small rotations.
- Neurology Context:** Real-time Brain-Computer Interfaces (BCIs). BCIs require incredibly fast processing. Can we strip away minor quantum gates to speed up execution without losing the ability to detect alpha or beta brainwaves?
- How to Code It:**
 - Generate a synthetic EEG signal (e.g., a combination of a 10 Hz alpha wave and a 20 Hz beta wave).
 - Encode a small window of this data (e.g., 8 or 16 data points) into a 3-qubit or 4-qubit quantum state using **Amplitude Encoding**.
 - Write a custom QFT function where you include an adjustable parameter `bandwidth_cutoff`. If an R_k gate's rotation angle is smaller than this cutoff, the code skips it.
 - Compare the resulting quantum frequency spectrum against a classical FFT.
- What you measure (Optimization Parameters):** Circuit Depth vs. Frequency Reconstruction Accuracy.
- Why it's Easiest:** Qiskit has built-in QFT modules, but writing a modified loop for a 3-to-4 qubit circuit is only a few lines of basic Python.

Recommended Choice for your Project: Experiment 1 or 2

If you want a highly successful project completed smoothly in 2 weeks, **Experiment 1 (Gate-Truncation)** is the sweet spot. *It directly answers the prompt: it optimizes a critical QFT parameter (rotation bandwidth/circuit depth), shows how that parameter affects a biological requirement (resolving specific brainwave frequencies), and can be written in less than 60 lines of cleanly commented Qiskit code on qBraid.*

- **Goal:** Tie the experiment back to your overarching research prompt.
- **Actions:**
 1. Write down your conclusion: *Did gate truncation preserve the brainwave features while lowering circuit depth?* (This directly answers what parameters are important for QFT optimization in neurology).
 2. Note how many gates you saved.
 3. Prepare notes for your future project: Because you now know how deleting gates changes the circuit's ideal performance, you are perfectly positioned to introduce a simulated IBM hardware noise model next time to see if this truncation trick actually protects the brainwave data from decoherence! (ties back into Experiment 3).

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Prompt: "When I build this project what kind of plots, graphs, or other metrics can I output that would support a research presentation?"

To support a research presentation on your Quantum Fourier Transform (QFT) optimization project, your visual assets need to tell a clear, concise data story: **How did manipulating the quantum circuit architecture affect the biological fidelity of your neurological data?** A compelling presentation will need a mix of circuit architecture visualizations, signal processing plots, and performance benchmarks. The specific outputs, graphs, and metrics to generate on qBraid include the following:

1. Circuit Structural Comparison (The "Before and After")

Before showing numerical results, you must visually establish what parameter you optimized.

- **The Plot:** A side-by-side or top-to-bottom structural view of your **Standard 3-Qubit QFT** vs. your **Truncated 3-Qubit QFT**.
- **How to get it in Qiskit:** Use `circuit.draw(output='mpl')` to generate crisp, publication-quality vector graphics of your circuits rather than text diagrams.
- **Presentation Value:** This provides an immediate visual anchor for your audience. You can explicitly highlight or circle the microscopic $\$CP(\pi/4)\$$ gate that you eliminated in the truncated version.

2. Time-Domain vs. Frequency-Domain Representation

Your audience needs to understand what the brain data looked like before and after entering the quantum pipeline.

- **The Plot:** A dual-panel figure:
 - **Panel A:** A standard continuous time-series line plot showing your blended synthetic EEG signal (the combined 7 Hz Theta and 15 Hz Beta waves), with markers indicating the exact 8 discrete points you sampled.
 - **Panel B:** A bar chart or line plot of a classical Fast Fourier Transform (FFT) of that exact data.
- **Metrics to include:** Sampling rate ($\$H_z\$\$), window length ($\$N=8\$\$).
- **Presentation Value:** This acts as your baseline proof-of-concept. It demonstrates that the classical signal processing math functions correctly before you introduce quantum mechanics.

3. The Core Result: Probability Distribution Histograms

This is the single most important plot of your presentation. It proves whether your optimized circuit successfully processed the neuroscience data.

- **The Plot:** A side-by-side or overlaid histogram using Qiskit's `plot_histogram`.
 - Bar set 1: Measured state probabilities from the **Standard QFT**.
 - Bar set 2: Measured state probabilities from the **Truncated QFT**.
- **What to look for:** In a 3-qubit system, your outcomes map to binary states ($\$|000\rangle\$$ to $\$|111\rangle\$\$), which translate to specific frequency bins. A successful project will show that *both* histograms have their highest peaks at the exact same states (the bins representing 7 Hz and 15 Hz), proving that truncation didn't destroy the neural feature detection.

4. Quantitative Performance Metrics (The "Proof of Optimization")

To confidently answer your research prompt ("What are the most important parameters..."), you must present concrete numbers showing what you gained by optimizing the code.

Create a **Summary Metrics Table** or a small bar chart displaying:

1. **Circuit Depth:** Calculate this using `circuit.depth()`. Show the percentage reduction in depth achieved by removing the small rotation gate. (*Lower circuit depth is the holy grail for Near-Term Quantum applications because it minimizes error*).
2. **Total Gate Count:** Use `circuit.count_ops()` to present the exact breakdown of gates (e.g., how many Hadamards vs. Controlled-Phase gates were used in each).
3. **Fidelity/Similarity Score (Bhattacharyya Coefficient or Hellinger Distance):** * **The Metric:** Use Qiskit's built-in quantum information module (`qiskit.quantum_info.hellinger_distance`) to calculate the statistical distance between your control distribution and your truncated distribution.
 - **Presentation Value:** If your Hellinger distance is close to 0, you can mathematically declare: "We reduced circuit complexity by X% while maintaining a Y% statistical match to the un-optimized quantum ideal."

Suggested Presentation Layout for Your Slides:

- **Slide 1: Introduction & Prompt** (Why brainwaves need fast Fourier processing).
- **Slide 2: Methodology** (The Time-Domain EEG plot + your Amplitude Encoding explanation).
- **Slide 3: Circuit Design** (The mpl circuit diagrams showing the gate removal).
- **Slide 4: Results** (The side-by-side Histogram showing matching frequency peaks).
- **Slide 5: Engineering Metrics** (The table comparing Circuit Depth reduction vs. State Fidelity).
- **Slide 6: Conclusion & Future Work** (Directly answering your prompt and teasing **Experiment 3: Hardware Noise Mitigation** as your next academic step!).

FFT Code

Tuesday, June 16, 2026 9:15 AM

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Here is the complete codebase for your **Quantum EEG Gate-Truncation Experiment**. This code is written in Python using Qiskit, NumPy, and Matplotlib. It handles the entire pipeline: generating synthetic neurological signals, mapping them to qubits via amplitude encoding, executing both the standard and optimized (truncated) quantum circuits, and calculating the final comparative metrics for your presentation.

1. Suggested File Structure

When organizing this project in your qBraid IDE directory, follow this modular structure. This makes your project look professional and easy to present.

Plaintext:

```
quantum_eeg_project/
├── main.py           # The master script that ties everything together
├── components/
│   ├── __init__.py   # Makes the folder an importable Python module
│   ├── signal_generator.py # Handles synthetic EEG wave creation
│   ├── quantum_circuits.py # Constructs standard and truncated QFTs
│   └── metrics_analyzer.py # Computes Hellinger distance, gate counts, and plots
```

2. Component Breakdowns & Code

You can create these files individually inside a folder named components/ on your qBraid workspace.

Component A: components/signal_generator.py

This script models the neuroscience aspect. It generates a mock brainwave (a combination of a 7 Hz Alpha/Theta wave and a 15 Hz Beta wave) and extracts exactly 8 normalized data points to feed into a 3-qubit system (\$2^3 = 8\$).

Python:

```
import numpy as np
def generate_eeg_signal(sampling_rate=128, duration=0.0625):
    """
    Generates a synthetic EEG signal combining 7 Hz and 15 Hz oscillations,
    and returns 8 normalized amplitude data points.
    """
    # 128 Hz sampling rate for 0.0625 seconds gives exactly 8 points
    t = np.linspace(0, duration, int(sampling_rate * duration), endpoint=False)

    # Simulate an organic brainwave (7 Hz wave + 15 Hz wave)
    wave_7hz = np.sin(2 * np.pi * 7 * t)
    wave_15hz = np.sin(2 * np.pi * 15 * t)
    raw_signal = wave_7hz + wave_15hz

    # AMPLITUDE ENCODING REQUIREMENT: Vector must be normalized (sum of squares = 1)
    norm_factor = np.linalg.norm(raw_signal)
    normalized_signal = raw_signal / norm_factor

    return t, raw_signal, normalized_signal
```

Component B: components/quantum_circuits.py

This script handles the quantum architecture. Instead of using Qiskit's high-level prebuilt QFT class, we manually build a 3-qubit QFT circuit. This allows us to explicitly truncate (delete) the smallest phase rotation gate (\$\pi/4\$) to measure its impact on the output.

Python:

```
import numpy as np
from qiskit import QuantumCircuit
def build_qft_circuit(amplitudes, truncate=False):
    """
    Creates a 3-qubit quantum circuit initialized with the EEG data.
    Applies either a full standard QFT or a depth-optimized truncated QFT.
    """
    # 3 qubits represent 8 data points; 3 classical bits for measurement
    qc = QuantumCircuit(3, 3)

    # Step 1: Load the classical EEG data amplitudes into the quantum state
    qc.initialize(amplitudes)
    qc.barrier() # Visual separator in the circuit diagram

    # Step 2: Custom 3-Qubit QFT Routine
    # Qubit 2
    qc.h(2)
    qc.cp(np.pi / 2, 1, 2)

    # The smallest gate: CP(pi/4) between Qubit 0 and 2
    if not truncate:
        qc.cp(np.pi / 4, 0, 2) # Kept in the Control Group
    else:
        pass # Removed in the Experimental Group (Optimized)

    # Qubit 1
    qc.barrier()
    qc.h(1)
    qc.cp(np.pi / 2, 0, 1)

    # Qubit 0
    qc.barrier()
    -- h(0)
```

Location: Gemini - WerthIAF 2026 (Quantum Coding)

Here is a professional, complete **README.md** file for your project. It includes an overview of the experiment, structural requirements, installation steps, and the exact terminal commands required to run everything on qBraid. You can create a file named README.md in your project's root directory and copy-paste the text below:

Markdown:

```
# Quantum EEG Optimization: QFT Gate-Truncation Experiment
This repository contains a Python and IBM Qiskit framework designed to evaluate the optimization of Quantum Fourier Transform (QFT) code for applications in neurology (such as real-time Brain-Computer Interfaces). The project evaluates a Gate-Truncation Strategy: comparing a standard 3-qubit QFT (Control Group) against an optimized version where the smallest microscopic rotation gate (CP(\pi/4)) is removed (Experimental Group). It analyzes whether deleting this gate significantly reduces circuit depth while preserving the biological integrity of a synthetic EEG signal (7 Hz Theta and 15 Hz Beta waves).
---
## Project File Structure
Ensure your directory is arranged exactly like this on your qBraid workspace:
```text
quantum_eeg_project/
├── README.md # Project documentation and execution instructions
├── main.py # Master coordination script
├── components/
│ ├── __init__.py # Marks directory as an importable module
│ ├── signal_generator.py # Generates synthetic biological EEG data
│ ├── quantum_circuits.py # Constructs the standard and truncated QFTs
│ └── metrics_analyzer.py # Evaluates circuit physics and plots data
```

## Installation & Environment Setup

Before running the code, ensure your qBraid environment has the correct dependencies installed.

1. Open your terminal in the qBraid IDE.
2. Install the necessary packages by running the following command:

### Bash:

```
pip install qiskit qiskit-aer numpy matplotlib scipy
```

## Step-by-Step Execution Guide

Follow these commands in exact order to create the directories, move files, and execute the experiment.

### Step 1: Create the Project Directories

In your terminal, navigate to your workspace and create the project folder along with its internal components folder:

### Bash:

```
mkdir -p quantum_eeg_project/components
cd quantum_eeg_project
```

### Step 2: Create and Populate the Files

Create the blank files within your new directories. You can populate them by opening them inside the qBraid file explorer editor and pasting the corresponding Python code blocks provided in your codebase.

### Bash:

```
touch main.py
touch components/__init__.py
touch components/signal_generator.py
touch components/quantum_circuits.py
touch components/metrics_analyzer.py
```

### Step 3: Run the Experiment

Once the code components are completely pasted and saved, execute the master script from the root quantum\_eeg\_project/ directory:

### Bash:

```
python main.py
```

## Expected Presentation Outputs

Upon successful execution, the script will output critical data points and graphs to support your research presentation directly in your workspace environment:

1. **Text Metrics (Terminal Display):**
  - o Standard vs. Truncated Circuit Depth (expecting a significant structural reduction).
  - o Absolute Gate Counts highlighting the targeted removal of the Controlled-Phase gate.
  - o **Total Variation Distance (TVD):** A statistical measurement indicating how closely the optimized output matches the perfect quantum baseline.
2. **Visual Figures (Matplotlib/Qiskit Windows):**

```
qc.m(1)
qc.cp(np.pi / 2, 0, 1)
```

```
Qubit 0
qc.barrier()
qc.h(0)
```

```
Step 3: Swap gates to correct frequency ordering
qc.swap(0, 2)
qc.barrier()
```

```
Step 4: Add measurements
qc.measure(,)
```

```
return qc
```

### Component C: components/metrics\_analyzer.py

This file extracts engineering metrics like circuit depth, runs the simulator backend, and handles the visual data generation you will need for your research presentation.

#### Python:

```
import matplotlib.pyplot as plt
from qiskit.visualization import plot_histogram
from qiskit import transpile
Update import for Hellinger distance to compatible locations or custom computing
from scipy.spatial.distance import total_variation_distance
def evaluate_and_plot(time, raw_signal, qc_standard, qc_truncated, result_std, result_trunc):
```

```
 """
 Computes presentation-ready metrics and plots the experimental results.
 """
 # --- Print Engineering Metrics ---
 print("=====")
 print(" QUANTUM ARCHITECTURE OPTIMIZATION METRICS ")
 print("=====")
 print(f"Standard Circuit Depth: {qc_standard.depth()}")
 print(f"Truncated Circuit Depth: {qc_truncated.depth()}")
 depth_saved = ((qc_standard.depth() - qc_truncated.depth()) / qc_standard.depth()) * 100
 print(f"Circuit Depth Reduction: {depth_saved:.1f}%")
```

```
 # Total Gate Count (Operations)
 print(f"Standard Gate Counts: {dict(qc_standard.count_ops())}")
 print(f"Truncated Gate Counts: {dict(qc_truncated.count_ops())}")
 print("=====\\n")
```

```
--- Plot 1: Neural Signal Input ---
plt.figure(figsize=(8, 3))
plt.plot(time, raw_signal, '-o', color='purple', label='Continuous EEG Signal')
plt.title("Time-Domain Synthetic Brainwave Data (7 Hz + 15 Hz)")
plt.xlabel("Time (seconds)")
plt.ylabel("Voltage (arbitrary units)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
--- Plot 2: Quantum Output Histograms ---
Convert counts to probability distributions for distance calculation
Express comparison as total variation distance
counts_std = result_std.get_counts()
counts_trunc = result_trunc.get_counts()
```

```
all_states = [format(i, '03b') for i in range(8)]
prob_std = [counts_std.get(s, 0) / 1024 for s in all_states]
prob_trunc = [counts_trunc.get(s, 0) / 1024 for s in all_states]
tvd = total_variation_distance(prob_std, prob_trunc)
```

```
print(f"Statistical Distance (Total Variation Distance): {tvd:.4f}")
print("A value close to 0 proves the optimized circuit did not compromise frequency integrity!")
```

```
Display side-by-side quantum histogram histograms
display(plot_histogram([counts_std, counts_trunc],
 legend=['Standard QFT (Control)', 'Truncated QFT (Optimized)'],
 color=['#808080', '#008080']))
```

### 3. Master Coordination Script: main.py

This is your main execution script. It loads your custom components, passes the data through the pipelines, calls the AerSimulator, and outputs all of your metrics.

#### Python:

```
Import native simulation backends
from qiskit_aer import AerSimulator
Import your custom project modules
from components.signal_generator import generate_eeg_signal
from components.quantum_circuits import build_qft_circuit
from components.metrics_analyzer import evaluate_and_plot
def main():
 # 1. Generate biological signal data
 time, raw_signal, normalized_amplitudes = generate_eeg_signal()

 # 2. Build the control and optimized quantum circuits
 circuit_standard = build_qft_circuit(normalized_amplitudes, truncate=False)
 circuit_truncated = build_qft_circuit(normalized_amplitudes, truncate=True)

 # 3. Initialize the simulator backend
 backend = AerSimulator()

 # 4. Transpile and run the Standard QFT
 # On qBraid, transpile handles mapping abstract logic to simulated hardware
 compiled_std = transpile(circuit_standard, backend)
```

- o Absolute Gate Counts highlighting the targeted removal of the Controlled-Phase gate.
- o **Total Variation Distance (TVD):** A statistical measurement indicating how closely the optimized output matches the perfect quantum baseline.

#### 2. Visual Figures (Matplotlib/Qiskit Windows):

- o **Figure 1:** A time-domain line plot charting the squiggly 7 Hz and 15 Hz combined synthetic neural signal.
- o **Figure 2:** A side-by-side comparative histogram showcasing the probability outcomes of both circuits to prove frequency preservation.

```

job_std = backend.run(compiled_std, shots=1024)
result_std = job_std.result()

5. Transpile and run the Optimized Truncated QFT
compiled_trunc = transpile(circuit_truncated, backend)
job_trunc = backend.run(compiled_trunc, shots=1024)
result_trunc = job_trunc.result()

6. Analyze performance metrics and produce presentation graphics
evaluate_and_plot(time, raw_signal,
 circuit_standard, circuit_truncated,
 result_std, result_trunc)

if __name__ == "__main__":
 main()

```

## How to run this on qBraid:

1. Create a workspace folder on your qBraid file explorer.
2. Save the files exactly as laid out in the file structure above.
3. Open a terminal in qBraid, make sure your environment is activated, and run:  
Bash
 

```
python main.py
```
4. Alternatively, you can copy and paste these code cells consecutively down into a single Jupyter Notebook file if you prefer a single-scroll presentation workspace.

### The truncation

- 1) Method of dropping qubits and rotations for speed
- 2) Using q fft to look at frequency and learn it quicker

### Speed optimization method

Why would looking at increase in speed matter? What does it improve in the signal processing?

What is the driving force/benefit for this potential method?

# Week 4

Tuesday, June 2, 2026

12:11 PM

Day 16 (06/22) -

Day 17 (06/23) -

Day 18 (06/24) -

Day 19 (06/25) -

Day 20 (06/26) -

# Week 5 - Slow Week

Tuesday, June 2, 2026

12:11 PM

Day 21 (06/29) -

Day 22 (06/30) -

Day 23 - (07/01) -

Day 24 - (07/02) -

Day 25 - (07/03) -

# Week 6

Tuesday, June 2, 2026

12:11 PM

Day 26 (07/06) -

Day 27 (07/07) -

Day 28 (07/08) -

Day 29 (07/09) -

Day 30 (07/10)-

# Week 7 - Explanation of Code

Tuesday, June 2, 2026

12:11 PM

Day 31 (07/13) -

Day 32 (07/14) -

Day 33 (07/15) -

Day 34 (07/16) - Adding & Explaining Code  
Read Me! Etc.

Day 35 (07/17) -

# Read Me!

Thursday, July 16, 2026 2:42 PM

## ## Fresh Install Steps

1. Open vsCode and folder locations
2. Open Terminal
3. run "python3 -m venv .venv" in terminal
4. run the below
  - ".venv\Scripts\Activate.ps1" for Windows.
  - "source .venv/bin/activate" for macOS
5. run "pip install -r requirements.txt"
6. run "pip list" - to check that everything was installed correctly
7. run "python3 main.py"

## ## Useful Commands

deactivate - manually get out of the virtual environment

# Components

Thursday, July 16, 2026 2:43 PM

# Metrics Analyzer

Thursday, July 16, 2026 2:43 PM

```
import matplotlib.pyplot as plt
import numpy as np
from qiskit.visualization import plot_histogram
from qiskit import transpile
from qiskit import QuantumCircuit
import os
import re
from scipy.fft import fft, fftfreq
os.makedirs("figures", exist_ok=True)
Update import for Hellinger distance to compatible locations or custom computing
from scipy.spatial.distance import total_variation_distance

def evaluate_and_plot(time, raw_signal, qc_standard, qc_truncated, result_std, result_trunc,
 experiment_name, wave_7hz, wave_15hz):
 """
 Computes presentation-ready metrics and plots the experimental results.
 """
 # --- Print Engineering Metrics ---
 print("=====")
 print(" QUANTUM ARCHITECTURE OPTIMIZATION METRICS ")
 print("=====")
 print(f"Standard Circuit Depth: {qc_standard.depth()}")
 print(f"Truncated Circuit Depth: {qc_truncated.depth()}")
 depth_saved = ((qc_standard.depth() - qc_truncated.depth()) / qc_standard.depth()) * 100
 print(f"Circuit Depth Reduction: {depth_saved:.1f}%")

 # Total Gate Count (Operations)
 print(f"Standard Gate Counts: {dict(qc_standard.count_ops())}")
 print(f"Truncated Gate Counts: {dict(qc_truncated.count_ops())}")
 print("=====\n")

 # --- Text-based prinout of circuit diagrams. Need to work on qc_standard.draw (output="mpl") etc working
 # (7/2/2026)
 print(qc_standard)
 print(qc_truncated)

 # --- Plot 1: Neural Signal Input ---
 # Generate the side-by-side histogram figure
 plt.figure(figsize=(8, 3))
 plt.plot(time, raw_signal, '-o', color='purple', label='Continuous EEG Signal')
 plt.title(f"{experiment_name} - Time-Domain Synthetic Brainwave Data")
 plt.xlabel("Time (seconds)")
 plt.ylabel("Amplitude (arbitrary units)")
 plt.grid(True)
 plt.legend()
 plt.tight_layout()
 safe_name = experiment_name.lower().replace(" ", "_").replace("(", "").replace(")", "")
 plt.savefig(f"figures/{safe_name}_neural_signal.png")

 # --- Plot: Frequency Components of EEG Signal ---
 num_samples = len(raw_signal)
 # Compute FFT
 fft_values = fft(raw_signal)
 # Frequency axis
 freqs = fftfreq(
 num_samples,
 d=(time[1] - time[0])
)
 # Only keep positive frequencies
 positive_freqs = freqs[:num_samples // 2]
 positive_fft = np.abs(fft_values[:num_samples // 2])
 plt.figure(figsize=(8, 4))
 plt.plot(
 positive_freqs,
 positive_fft,
 color="purple",
 linewidth=2
)
```

```

plt.title(f"{experiment_name} - EEG Frequency Components")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(alpha=0.3)
Highlight expected frequency components
plt.axvline(
 7,
 color="orange",
 linestyle="--",
 label="7 Hz Component"
)
plt.axvline(
 15,
 color="green",
 linestyle="--",
 label="15 Hz Component"
)
plt.legend()
plt.tight_layout()
safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", experiment_name)
plt.savefig(
 f"figures/{safe_name}_frequency_components.png",
 dpi=300,
 bbox_inches="tight"
)
--- Plot: EEG Signal Components ---
fig, axes = plt.subplots(
 3,
 1,
 figsize=(10, 8),
 sharex=True
)
7 Hz component
axes[0].plot(
 time,
 wave_7hz,
 color="#F07021",
 linewidth=2
)
axes[0].set_title("7 Hz EEG Component")
axes[0].set_ylabel("Amplitude")
axes[0].grid(alpha=0.3)

15 Hz component
axes[1].plot(
 time,
 wave_15hz,
 color="#00806B",
 linewidth=2
)
axes[1].set_title("15 Hz EEG Component")
axes[1].set_ylabel("Amplitude")
axes[1].grid(alpha=0.3)

Combined EEG signal
axes[2].plot(
 time,
 raw_signal,
 color="purple",
 linewidth=2
)
axes[2].set_title("Combined Synthetic EEG Signal")
axes[2].set_xlabel("Time (seconds)")
axes[2].set_ylabel("Amplitude")
axes[2].grid(alpha=0.3)

fig.suptitle(
 f"{experiment_name} - EEG Signal Components",
 fontsize=14
)

```

```

plt.tight_layout()
safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", experiment_name)
plt.savefig(
 f"figures/{safe_name}_eeg_components.png",
 dpi=300,
 bbox_inches="tight"
)
--- Plot 2: Quantum Output Histograms ---
Convert counts to probability distributions for distance calculation
Express comparison as total variation distance
counts_std = result_std.get_counts()
counts_trunc = result_trunc.get_counts()
print(counts_std, counts_trunc)

all_states = [format(i, '03b') for i in range(8)]
prob_std = [counts_std.get(s, 0) / 1024 for s in all_states]
prob_trunc = [counts_trunc.get(s, 0) / 1024 for s in all_states]
tvd = 0.5 * np.sum(np.abs(np.array(prob_std) - np.array(prob_trunc)))
tvd = total_variation_distance(prob_std, prob_trunc)

print(f"Statistical Distance (Total Variation Distance): {tvd:.4f}")
print("A value close to 0 proves the optimized circuit did not compromise frequency integrity!")
Generate the side-by-side histogram figure
hist = plot_histogram(
 [counts_std, counts_trunc],
 legend=["Standard QFT (Control)", "Truncated QFT (Optimized)"],
 color=["#F07021", "#00806B"]
)
--- Add your labels and title here ---
ax = hist.axes[0]
ax.set_xlabel("Computational Basis States")
ax.set_ylabel("Count")
ax.set_title(f"{experiment_name} - Quantum Fourier Transform Output Comparison")

Save the histogram directly to your current project folder
hist.savefig(f"figures/{safe_name}_quantum_histogram.png")
print("Histogram saved successfully as 'quantum_histogram.png' to 'figures' folder.")
--- Plot 3: Connected Scatter Trend Comparison ---
Order computational states
all_states = sorted(set(counts_std.keys()) | set(counts_trunc.keys()))
Convert counts to probabilities
prob_std = [
 counts_std.get(state, 0) / 1024
 for state in all_states
]
prob_trunc = [
 counts_trunc.get(state, 0) / 1024
 for state in all_states
]
plt.figure(figsize=(10, 5))
plt.plot(
 all_states,
 prob_std,
 marker="o",
 linewidth=2.5,
 color="#F07021",
 label="Standard QFT (Control)"
)
plt.plot(
 all_states,
 prob_trunc,
 marker="s",
 linestyle="--",
 linewidth=2.5,
 color="#00806B",
 label="Truncated QFT (Optimized)"
)
plt.fill_between(
 all_states,
 prob_std,
 prob_trunc,

```

```

 color="gray",
 alpha=0.15
)
plt.title(f"{experiment_name} - QFT Output Probability Comparison")
plt.xlabel("Computational Basis State")
plt.ylabel("Measurement Probability")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", experiment_name)
plt.savefig(
 f"figures/{safe_name}_quantum_trend.png",
 dpi=300,
 bbox_inches="tight"
)
--- Show both plots after they are saved to figures folder ---
hist.show()
print("hist.show ran")
plt.show()
print("plt.show ran")

```

# Quantum Circuit

Thursday, July 16, 2026 2:45 PM

```
import numpy as np
from qiskit import QuantumCircuit

def build_qft_circuit_3qb(amplitudes, truncate=False):
 """
 Creates a 3-qubit quantum circuit initialized with the EEG data.
 Applies either a full standard QFT or a depth-optimized truncated QFT.
 """
 # 3 qubits represent 8 data points; 3 classical bits for measurement
 qc = QuantumCircuit(3, 3)

 # Step 1: Load the classical EEG data amplitudes into the quantum state
 # qc.initialize(amplitudes,)
 qc.initialize(amplitudes, [0, 1, 2])
 qc.barrier() # Visual separator in the circuit diagram

 # Step 2: Custom 3-Qubit QFT Routine
 # Qubit 2
 qc.h(2)
 qc.cp(np.pi / 2, 1, 2)

 # The smallest gate: CP(pi/4) between Qubit 0 and 2
 if not truncate:
 qc.cp(np.pi / 4, 0, 2) # Kept in the Control Group
 else:
 pass # Removed in the Experimental Group (Optimized)

 # Qubit 1
 qc.barrier()
 qc.h(1)
 qc.cp(np.pi / 2, 0, 1)

 # Qubit 0
 qc.barrier()
 qc.h(0)

 # Step 3: Swap gates to correct frequency ordering
 qc.swap(0, 2)
 qc.barrier()

 # Step 4: Add measurements
 # qc.measure(,)
 qc.measure([0, 1, 2], [0, 1, 2])
 return qc

def build_qft_circuit_5qb(amplitudes, truncate=False):
 """
 Creates a 5-qubit quantum circuit initialized with the EEG data.
 Applies either a full standard QFT or a depth-optimized truncated QFT.
 """
 # 5 qubits represent 32 data points; 5 classical bits for measurement
 qc = QuantumCircuit(5, 5)

 # Step 1: Load the classical EEG data amplitudes into the quantum state
 qc.initialize(amplitudes, [0, 1, 2, 3, 4])
 qc.barrier() # Visual separator in the circuit diagram

 # Step 2: Custom 5-Qubit QFT Routine

 # --- Qubit 4 ---
 qc.h(4)
 qc.cp(np.pi / 2, 3, 4)
 qc.cp(np.pi / 4, 2, 4)
 if not truncate:
 qc.cp(np.pi / 8, 1, 4) # Removed if truncate=True
 qc.cp(np.pi / 16, 0, 4) # Removed if truncate=True
```

```

qc.barrier()

--- Qubit 3 ---
qc.h(3)
qc.cp(np.pi / 2, 2, 3)
qc.cp(np.pi / 4, 1, 3)
if not truncate:
 qc.cp(np.pi / 8, 0, 3) # Removed if truncate=True
qc.barrier()

--- Qubit 2 ---
qc.h(2)
qc.cp(np.pi / 2, 1, 2)
qc.cp(np.pi / 4, 0, 2)
qc.barrier()

--- Qubit 1 ---
qc.h(1)
qc.cp(np.pi / 2, 0, 1)
qc.barrier()

--- Qubit 0 ---
qc.h(0)
qc.barrier()

Step 3: Swap gates to correct frequency ordering (Mirror inversion)
qc.swap(0, 4)
qc.swap(1, 3)
qc.barrier()

Step 4: Add measurements
qc.measure([0, 1, 2, 3, 4], [0, 1, 2, 3, 4])
return qc
def build_qft_circuit_7qb(amplitudes, truncate=False):
 """
 Creates a 7-qubit quantum circuit initialized with the EEG data.
 Applies either a full standard QFT or a depth-optimized truncated QFT.
 """
 qc = QuantumCircuit(7, 7)
 # Step 1: Load the classical EEG data amplitudes
 qc.initialize(amplitudes, list(range(7)))
 qc.barrier()
 # ----- Qubit 6 -----
 qc.h(6)
 qc.cp(np.pi/2, 5, 6)
 qc.cp(np.pi/4, 4, 6)
 if not truncate:
 qc.cp(np.pi/8, 3, 6)
 qc.cp(np.pi/16, 2, 6)
 qc.cp(np.pi/32, 1, 6)
 qc.cp(np.pi/64, 0, 6)
 qc.barrier()
 # ----- Qubit 5 -----
 qc.h(5)
 qc.cp(np.pi/2, 4, 5)
 qc.cp(np.pi/4, 3, 5)
 if not truncate:
 qc.cp(np.pi/8, 2, 5)
 qc.cp(np.pi/16, 1, 5)
 qc.cp(np.pi/32, 0, 5)
 qc.barrier()
 # ----- Qubit 4 -----
 qc.h(4)
 qc.cp(np.pi/2, 3, 4)
 qc.cp(np.pi/4, 2, 4)
 if not truncate:
 qc.cp(np.pi/8, 1, 4)
 qc.cp(np.pi/16, 0, 4)
 qc.barrier()
 # ----- Qubit 3 -----
 qc.h(3)

```

```

qc.cp(np.pi/2, 2, 3)
qc.cp(np.pi/4, 1, 3)
if not truncate:
 qc.cp(np.pi/8, 0, 3)
qc.barrier()
----- Qubit 2 -----
qc.h(2)
qc.cp(np.pi/2, 1, 2)
qc.cp(np.pi/4, 0, 2)
qc.barrier()
----- Qubit 1 -----
qc.h(1)
qc.cp(np.pi/2, 0, 1)
qc.barrier()
----- Qubit 0 -----
qc.h(0)
qc.barrier()
Swap qubits
qc.swap(0, 6)
qc.swap(1, 5)
qc.swap(2, 4)
qc.barrier()
Measurements
qc.measure(range(7), range(7))
return qc

```

# Signal Generator

Thursday, July 16, 2026 2:45 PM

```
import numpy as np
This code will generate a sinusoidal wave form and descrtize it in either 8 or 32
data points. The more datapoints, the more qubits required to encode.
def generate_eeg_signal_8(sampling_rate=128, duration=0.0625):
 """
 Generates a synthetic EEG signal combining 7 Hz and 15 Hz oscillations,
 and returns 8 normalized amplitude data points.
 """
 # 128 Hz sampling rate for 0.0625 seconds gives exactly 8 points
 t = np.linspace(0, duration, int(sampling_rate * duration), endpoint=False)

 # Simulate an organic brainwave (7 Hz wave + 15 Hz wave)
 wave_7hz = np.sin(2 * np.pi * 7 * t)
 wave_15hz = np.sin(2 * np.pi * 15 * t)
 raw_signal = wave_7hz + wave_15hz

 # AMPLITUDE ENCODING REQUIREMENT: Vector must be normalized (sum of squares = 1)
 norm_factor = np.linalg.norm(raw_signal)
 normalized_signal = raw_signal / norm_factor

 return t, raw_signal, normalized_signal, wave_7hz, wave_15hz
def generate_eeg_signal_32(sampling_rate=512, duration=0.0625):
 """
 Generates a synthetic EEG signal combining 7 Hz and 15 Hz oscillations,
 and returns 32 normalized amplitude data points for 5-qubit state initialization.
 """
 # 512 Hz sampling rate for 0.0625 seconds gives exactly 32 points
 num_points = int(sampling_rate * duration)
 t = np.linspace(0, duration, num_points, endpoint=False)

 # Simulate an organic brainwave (7 Hz wave + 15 Hz wave)
 wave_7hz = np.sin(2 * np.pi * 7 * t)
 wave_15hz = np.sin(2 * np.pi * 15 * t)
 raw_signal = wave_7hz + wave_15hz

 # AMPLITUDE ENCODING REQUIREMENT: Vector must be normalized (sum of squares = 1)
 norm_factor = np.linalg.norm(raw_signal)
 normalized_signal = raw_signal / norm_factor

 return t, raw_signal, normalized_signal, wave_7hz, wave_15hz
def generate_eeg_signal_128(sampling_rate=512, duration=0.25):
 """
 Generates a synthetic EEG signal combining 7 Hz and 15 Hz oscillations,
 and returns 128 normalized amplitude data points for 7-qubit state initialization.
 """
 # 512 Hz sampling rate for 0.25 seconds gives exactly 128 points
 num_points = int(sampling_rate * duration)
 t = np.linspace(0, duration, num_points, endpoint=False)
 # Simulate an organic brainwave (7 Hz + 15 Hz)
 wave_7hz = np.sin(2 * np.pi * 7 * t)
 wave_15hz = np.sin(2 * np.pi * 15 * t)
 raw_signal = wave_7hz + wave_15hz
 # Normalize for amplitude encoding
 norm_factor = np.linalg.norm(raw_signal)
 normalized_signal = raw_signal / norm_factor
 return t, raw_signal, normalized_signal, wave_7hz, wave_15hz
```

# Main

Thursday, July 16, 2026 2:46 PM

```
Import native simulation backends
from qiskit_aer import AerSimulator
from qiskit import transpile
Import your custom project modules
from components.signal_generator import (
 generate_eeg_signal_8,
 generate_eeg_signal_32,
 generate_eeg_signal_128,
)
from components.quantum_circuits import (
 build_qft_circuit_3qb,
 build_qft_circuit_5qb,
 build_qft_circuit_7qb,
)
from components.metrics_analyzer import evaluate_and_plot
import os
import matplotlib.pyplot as plt
import re
os.makedirs("figures", exist_ok=True)
def run_experiment(title, signal_generator, circuit_builder):
 print(f"\n{'=' * 60}")
 print(title)
 print(f"{'=' * 60}")
 # Generate biological signal
 time, raw_signal, normalized_amplitudes, wave_7hz, wave_15hz = signal_generator()
 # Build circuits
 circuit_standard = circuit_builder(normalized_amplitudes, truncate=False)
 circuit_truncated = circuit_builder(normalized_amplitudes, truncate=True)
 # Print Circuits
 print("\nStandard QFT Circuit:")
 # Print the exact gate breakdown for your Standard QFT
 print("Standard QFT Gates:", circuit_standard.count_ops())
 print(circuit_standard)
 print("\nTruncated QFT Circuit:")
 # Print the exact gate breakdown for your Truncated QFT
 print("Truncated QFT Gates:", circuit_truncated.count_ops())
 print(circuit_truncated)
 safe_name = re.sub(r"[^A-Za-z0-9_-]+", "_", title)
 fig = circuit_standard.draw(output="mpl", fold=-1)
 fig.savefig(
 f"figures/{safe_name}_standard_qft_circuit.png",
 dpi=300,
 bbox_inches="tight"
)
 plt.close(fig)
 fig = circuit_truncated.draw(output="mpl", fold=-1)
 fig.savefig(
 f"figures/{safe_name}_truncated_qft_circuit.png",
 dpi=300,
 bbox_inches="tight"
)
 plt.close(fig)
 # Simulator
 backend = AerSimulator()
 # Run Standard QFT
 compiled_std = transpile(circuit_standard, backend)
 result_std = backend.run(compiled_std, shots=1024).result()
 # Run Truncated QFT
 compiled_trunc = transpile(circuit_truncated, backend)
 result_trunc = backend.run(compiled_trunc, shots=1024).result()
 # Analyze results
 evaluate_and_plot(
 time,
 raw_signal,
 circuit_standard,
 circuit_truncated,
```

```

 result_std,
 result_trunc,
 title,
 wave_7hz,
 wave_15hz
)

def main():
 # Experiment 1: 8-point EEG using 3 qubits
 run_experiment(
 "8-Sample EEG (3-Qubit QFT)",
 generate_eeg_signal_8,
 build_qft_circuit_3qb,
)
 # Experiment 2: 32-point EEG using 5 qubits
 run_experiment(
 "32-Sample EEG (5-Qubit QFT)",
 generate_eeg_signal_32,
 build_qft_circuit_5qb,
)
 # Experiment 3: 128-point EEG using 5 qubits
 run_experiment(
 "128-Sample EEG (7-Qubit QFT)",
 generate_eeg_signal_128,
 build_qft_circuit_7qb,
)
if __name__ == "__main__":
 main()

```

# Week 8

Tuesday, June 2, 2026

12:11 PM

Day 36 (07/20) -

Day 37 (07/21)-

Day 38 (07/22) -

Day 39 (07/23) -

Day 40 (07/24) -